

TradeMirror — AI Positioning Tracking & Learning Plan

Implementation reference for daily AI positioning snapshots, outcome tracking, and the eventual reinforcement / machine learning loop. Self-contained: any future Claude Code session should be able to execute this without prior context.

1. Feature Requirements (Geoff's spec)

- **Daily snapshot** at ~6pm local time. The most recent daily OANDA candle has just closed (5pm NY), so this is the first moment fresh daily data is available.
- **Validity:** the snapshot is the active AI positioning for the next 24 hours, until the next day's snapshot replaces it.
- **Day-trading tuned:** entries / stops / TPs sized for an intraday holding window — not swing.
- **Outcome tracking:** every snapshot must be resolved against subsequent price action. Did TP1 / TP2 / TP3 hit? Did SL hit? What was max favourable / adverse excursion?
- **Goal metric:** maximise positive P&L over a rolling window. Track win rate, R-multiple expectancy, and per-condition breakdowns.
- **Learning loop:** use accumulated outcomes to tune scoring weights and, eventually, train an ML calibration model on top of the deterministic features.
- **Multi-pair architecture:** EUR/USD is phase 1, but every table / command / weight set must be keyed by pair from day one.

2. Current TradeMirror State (build on what exists)

As of this document's creation, the following infrastructure is already in place. The new feature consumes these — do not duplicate.

Centralised scoring engine — src-tauri/src/scoring/

- **mod.rs** — Synthesis, ScoreBlock, SignalContribution, Rationale, MultiTimeframe, RegimeContext types; composite aggregator with regime weights; build_synthesis() orchestrator.
- **trend.rs** — signed trend sub-score: EMA stack, SMA200, MACD line, ADX+DI, Ichimoku, Avg Price.
- **momentum.rs** — signed momentum sub-score: RSI(14) zone, StochRSI, MACD hist, CCI, Williams %R, Momentum(10), ROC(5), Avg Delta.
- **structure.rs** — market structure with ATR displacement filter, real-time event detection, 6-tier state (Strong Bullish / Bullish / Shifting / Range / Bearish / Strong Bearish).
- **volatility.rs** — 5-tier regime classifier (StrongTrending / Trending / Expansion / Ranging / Compression) with joint BB+ATR conditions and DI dominance gate.

Tauri command currently exposed

get_synthesis(pair: String, tf: String) -> Synthesis in src-tauri/src/lib.rs. Fetches D + H4 + H1 candles for the given pair, scores each, returns the full Synthesis including rationale[], multi_timeframe, and context: RegimeContext.

Frontend bridge

- src/lib/brain/synthesis.ts — TS mirror of Synthesis types, fetchSynthesis() wrapper, synthesisToAnalysisResult() mapper for the AI Synthesis panel.
- AI Synthesis panel in src/pages/AnalyticsV3.tsx currently consumes synthesis on app load and on pair change (no auto-refresh).

- Positioning today: entry = current daily close, stop = entry $\pm 1.5 \times \text{ATR}(14)$ in price units, TPs = pivot R1/R2/R3 or S1/S2/S3.

Persistence — already present

- SQLite via tauri-plugin-sql + Drizzle ORM on the frontend.
- Migrations live in src-tauri/migrations/.
- candles_v3 table already stores daily candles with all indicator columns (0002_candles_v3.sql).
- src-tauri/src/candle_store.rs handles candle reads/writes.

Architecture rules (from CLAUDE.md — must follow)

- ONE candle source, ONE indicator engine, ONE structured output layer. Do not duplicate.
- Backend in Rust, UI consumes via Tauri commands. Don't compute indicators in the UI.
- Replay consistency: bar i never reads $i+1$.
- Validation before prediction; ML is not phase 1.
- Local-first, no paid services, existing stack only (Tauri, React, SQLite, Drizzle).

3. Architecture Overview

Five subsystems, each independent and testable:

- 1 **Scheduler** — fires once per day per active pair at 18:00 local time. Calls the snapshot writer.
- 2 **Snapshot writer** — runs `build_synthesis()` for the pair, derives day-trading-tuned positioning, writes a row to `positioning_snapshots`.
- 3 **Outcome resolver** — periodic background task (e.g. hourly). For every open snapshot whose `valid_until` has passed without resolution, walks subsequent intraday bars (15m or 1H) and records first-level-hit + MFE/MAE in `positioning_outcomes`.
- 4 **Reporting views** — SQL queries / UI panel that surface win rate, expectancy, per-regime / per-confidence-band breakdowns. This is the human-facing 'is there edge?' check.
- 5 **Weight tuner (later)** — once enough outcomes are recorded per slice, automated suggestions for tuning per-signal weights in scoring/*. Manual review before deploy.

Data flow:

```
OANDA → candles_v3 → indicators.rs::compute → scoring::build_synthesis →
positioning_writer (day-trading tuning applied here) → positioning_snapshots → [24h later]
outcome_resolver → positioning_outcomes → reporting_views → eventual_weight_tuner.
```

4. Database Schema

Two new SQLite tables. Both keyed by pair from day one for multi-pair support.

Table: `positioning_snapshots`

One row per (pair, day). Captures the full Synthesis state used to derive positioning, plus the positioning itself. The Synthesis rationale is stored as a JSON blob for RAG / post-hoc analysis.

```
CREATE TABLE positioning_snapshots (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  created_at TEXT NOT NULL, -- ISO 8601, UTC
  valid_until TEXT NOT NULL, -- created_at + 24h
  pair TEXT NOT NULL, -- e.g. 'EUR/USD'
  tf TEXT NOT NULL, -- '1D' for now
  direction TEXT NOT NULL, -- 'LONG' | 'SHORT' | 'NEUTRAL'
  confidence REAL NOT NULL, -- 0..100
  composite REAL NOT NULL, -- signed -100..+100
  trend_score REAL NOT NULL,
  momentum_score REAL NOT NULL,
  structure_score REAL NOT NULL,
  regime TEXT NOT NULL, -- StrongTrending | Trending | Expansion | Ranging | Compression
  volatility TEXT NOT NULL, -- Expanding | Neutral | Contracting
  mtf_alignment REAL NOT NULL, -- 0..100
  entry REAL NOT NULL,
  stop_loss REAL NOT NULL,
  tp1 REAL NOT NULL,
  tp2 REAL NOT NULL,
  tp3 REAL NOT NULL,
  risk_pips REAL NOT NULL, -- |entry - stop_loss| in pips
  rationale_json TEXT NOT NULL, -- full Synthesis.rationale[] as JSON
  synthesis_json TEXT NOT NULL, -- full Synthesis object for replay
  UNIQUE (pair, created_at)
);
CREATE INDEX idx_snap_pair_date ON positioning_snapshots(pair, created_at);
```

Table: `positioning_outcomes`

```

CREATE TABLE positioning_outcomes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  snapshot_id INTEGER NOT NULL REFERENCES positioning_snapshots(id),
  resolved_at TEXT NOT NULL,
  first_level_hit TEXT NOT NULL, -- 'tp1' | 'tp2' | 'tp3' | 'sl' | 'none'
  bars_to_resolution INTEGER NOT NULL, -- count of intraday bars from snapshot to hit
  mfe_pips REAL NOT NULL, -- max favourable excursion in pips
  mae_pips REAL NOT NULL, -- max adverse excursion in pips
  r_multiple REAL NOT NULL, -- P&L in R units (R = risk_pips)
  intraday_tf TEXT NOT NULL, -- '15M' or '1H', whichever was used
  notes TEXT,
  UNIQUE (snapshot_id)
);

```

first_level_hit semantics: walk intraday bars chronologically; the first bar whose range crosses any of TP1/TP2/TP3/SL determines the outcome. Ties (bar contains both) resolved as SL hit (conservative). 'none' means snapshot expired without any level hit.

r_multiple: $(\text{hit_level_price} - \text{entry}) / (\text{entry} - \text{stop_loss})$, signed by direction. TP1 $\approx +1R$ if pivots and stop are aligned; SL = $-1R$ by definition.

5. Day-Trading Positioning Derivation

Current synthesisToAnalysisResult uses $1.5 \times \text{ATR}(14)$ stops and pivot R/S levels. For day trading, tighten this so positions resolve within the 24h window.

Recommended rules (override the current positioning() helper for daily snapshots)

- **Entry:** daily close at snapshot time (already the case).
- **Stop loss:** $0.75 \times \text{ATR}(14)$ instead of $1.5 \times$ — tighter, suitable for intraday hold. Bound: minimum 15 pips, maximum 60 pips for EUR/USD-style majors.
- **TP1:** closest pivot in direction (R1 for long, S1 for short) — typically the most-frequently-hit intraday target.
- **TP2:** 1.5R from entry — a deterministic target based on risk size, not pivot dependent.
- **TP3:** 2.5R from entry — stretch target.
- **Risk filter:** if confidence < 25 OR regime = Compression OR mtf_alignment < 33, skip the snapshot (write a row with direction='NEUTRAL' and no positioning). Compression breakouts are unpredictable; low alignment is low-edge.

Place this in a new src-tauri/src/scoring/positioning.rs module that owns the day-trading derivation.

build_synthesis() already has all inputs needed; positioning is a thin wrapper that consumes the resulting Synthesis.

Rust signature

```
pub struct DayTradePositioning {
    pub direction: String, // LONG | SHORT | NEUTRAL
    pub entry: f64,
    pub stop_loss: f64,
    pub tp1: f64,
    pub tp2: f64,
    pub tp3: f64,
    pub risk_pips: f64,
    pub skipped: bool, // true if risk filter rejected
    pub skip_reason: Option<String>,
}

pub fn derive(synth: &Synthesis, atr_pips: f64, pip_size: f64) -> DayTradePositioning;
```

pip_size must be passed because pip definitions differ: EUR/USD = 0.0001, USD/JPY = 0.01, GBP/JPY = 0.01, etc. Caller derives from the pair string. Keep this in a small pip_size_for(pair: &str) -> f64 helper.

6. Scheduling — 18:00 Local Daily

OANDA's daily candle closes at 17:00 New York (22:00 UTC during EDT, 21:00 UTC during EST). Geoff wants the snapshot ~18:00 local — one hour after the new daily bar is available. On Tauri/Windows the cleanest approach is a **tokio background task** spawned at app startup.

Rust implementation sketch

```
// In src-tauri/src/lib.rs run() — spawn after app setup
tauri::async_runtime::spawn(async move {
    loop {
        let now = chrono::Local::now();
        let next = compute_next_1800(now);
        let wait = (next - now).to_std().unwrap_or_default();
        tokio::time::sleep(wait).await;
        for pair in active_pairs() {
            run_snapshot_for(&pair).await;
        }
    }
})
```

```
});
```

active_pairs() reads from a config table or a hardcoded list initially (['EUR/USD']). **run_snapshot_for** performs: fetch candles → build synthesis → derive positioning → insert row into positioning_snapshots. Also expose a Tauri command `force_snapshot(pair)` for manual triggering during testing.

Crash-tolerance: on app start, if no snapshot exists for today and current local time is > 18:00, immediately run a catch-up snapshot. Otherwise sleep until next 18:00.

7. Outcome Resolution Pipeline

Every open snapshot (no outcome row, `valid_until ≤ now`) must be resolved by walking intraday bars. Use H1 candles by default (288 max bars over 24h is unnecessary precision; H1 gives 24 bars and matches typical day-trading granularity).

Algorithm

- 1 Query positioning_snapshots WHERE id NOT IN (SELECT snapshot_id FROM positioning_outcomes) AND valid_until ≤ now().
- 2 For each snapshot: fetch H1 candles for pair covering [created_at, valid_until].
- 3 If direction = NEUTRAL: record outcome with first_level_hit='none', r_multiple=0, MFE=MAE=0. Skip the walk.
- 4 Otherwise walk bars chronologically. Track running MFE and MAE in pips.
- 5 For each bar, check if bar.high ≥ TP-level (for longs) or bar.low ≤ TP-level (for shorts). Same for SL. First level crossed wins.
- 6 If bar crosses both SL and a TP on the same bar (gap or wide range): record as SL hit (conservative).
- 7 If no level hit by end of window: first_level_hit='none', r_multiple = (final_close - entry) / risk_in_price * direction_sign.
- 8 Insert row into positioning_outcomes.

Run this resolver on a 60-minute tokio interval. Idempotent — won't re-resolve completed snapshots.

Tauri commands needed

```
fn list_open_snapshots() -> Vec<Snapshot>
fn list_resolved_snapshots(pair: String, limit: u32) -> Vec<ResolvedSnapshot>
fn force_resolve(snapshot_id: i64) -> Result<Outcome, String>
fn force_snapshot(pair: String) -> Result<i64, String>
fn get_accuracy_stats(pair: String, days: u32) -> AccuracyStats
```

8. Reporting Views & UI

New 'AI Positioning' page or panel

- Header: today's snapshot — direction, confidence, entry/SL/TP1/TP2/TP3, regime, rationale top 3.
- Body: recent history table — last 30 snapshots with their outcomes (TP1/TP2/TP3/SL/none), R-multiple, MFE/MAE.
- Aggregate metrics tile: win rate, expectancy (R), avg MFE, avg MAE, total R over window.
- Slice breakdowns: per-regime (rows of metrics), per-confidence-band (low/mid/high), per-MTF-alignment-band, per top-driver signal_id.

AccuracyStats Rust struct

```
pub struct AccuracyStats {  
    pub pair: String,  
    pub window_days: u32,  
    pub n_snapshots: u32,  
    pub n_resolved: u32,  
    pub n_skipped: u32,  
    pub win_rate: f64, // proportion of resolved where r_multiple > 0  
    pub expectancy_r: f64, // mean r_multiple across resolved  
    pub total_r: f64,  
    pub avg_mfe: f64,  
    pub avg_mae: f64,  
    pub by_regime: Vec<SliceStats>,  
    pub by_confidence: Vec<SliceStats>,  
    pub by_mtf_alignment: Vec<SliceStats>,  
    pub by_top_signal: Vec<SliceStats>,  
}
```

9. Multi-Pair Architecture

Geoff's instruction: design for multi-pair from day one, even though EUR/USD is the only active pair at build time.

- Every table row includes a pair column. No pair-implicit logic.
- `active_pairs()` reads a configurable list (table `tracked_pairs` or app config file). Initially returns ['EUR/USD']; expansion is config-only, no code change.
- `pip_size_for(pair)` handles JPY pairs (0.01) vs majors (0.0001) vs metals (0.1).
- Indicator engine is already pair-agnostic — same signals work for every instrument.
- Per-pair weights come later: introduce a `pair_weight_overrides` table once the validation phase shows different pairs require different scoring profiles. Until then all pairs share the default weights.
- Scheduler iterates all active pairs sequentially at 18:00; each pair takes ~3 OANDA calls (D + H4 + H1) so the full sweep is <30 seconds for ~5 pairs.
- Reporting UI: pair selector at top filters all aggregates.

10. Validation Phase (NO ML)

Per CLAUDE.md — 'validation before prediction; ML is not phase 1'. The first 60–90 days of snapshots are pure observation. Do not touch weights.

- **Minimum sample:** 30 resolved snapshots per pair before *any* conclusions; 90+ before tuning anything.

- **Track religiously:** win rate, expectancy in R, max consecutive loss streak, max drawdown in R.
- **Slice analysis:** are certain regimes producing all the wins? Is high MTF alignment correlated with higher R? Does Compression regime add or destroy value?
- **Honesty check:** a system with win_rate > 50% but expectancy ≤ 0 means TP1 hits often but losses are bigger than wins — fix sizing before declaring edge.
- **Out-of-sample test:** reserve the most recent 30 snapshots as a holdout. Tune on the rest, validate on the holdout. If holdout expectancy collapses, the tuning overfit.

The validation phase outcome should be one of three verdicts:

- 1 **Edge confirmed** — expectancy > +0.2R, win rate > 35%, holdout matches. Proceed to weight tuning.
- 2 **Edge unclear** — expectancy near 0, mixed slice signals. Either continue collecting or revisit signal selection.
- 3 **No edge** — expectancy ≤ 0 across slices, holdout confirms. Re-design positioning logic before tuning.

11. Weight Tuning & Eventual ML

Phase A — Manual weight tuning (post-validation, pre-ML)

- For each signal_id in the rationale, compute conditional expectancy: 'when this signal contributed > X to the composite, what was the resolved R-multiple?'
- Signals with positive conditional expectancy → increase their value magnitude in scoring/{trend,momentum,structure}.rs.
- Signals with neutral / negative → reduce or zero out.
- **Shadow run:** store proposed weights in a weight_proposals table; the scheduler can run both production and shadow weights in parallel for 30 days before promotion.
- Promote only when shadow expectancy beats production by > 0.1R on out-of-sample data.

Phase B — ML calibration layer (much later)

- Inputs: the structured Synthesis features (sub-scores, regime one-hot, MTF alignment, per-signal contributions, ATR-normalised price location vs pivots).
- Target: probability that r_multiple > +1 within 24h.
- Model: gradient-boosted trees (XGBoost) or a small MLP. Lightweight, runs in Rust via ONNX or tract.
- Output: a calibration multiplier applied to the deterministic confidence — never replaces the deterministic decision, only refines its strength.
- **Hard rule:** ML output must never bypass risk filters. If deterministic says skip, ML cannot un-skip.
- **Re-train cadence:** weekly or after every 50 new resolved snapshots. Always evaluate against the same holdout to detect regression.

Phase C — Reinforcement learning (research only)

Once both Phase A and Phase B show stable edge, RL can experiment with adaptive position sizing (not entry/stop logic — that's the deterministic system's job). RL inputs: current drawdown, recent win streak, regime, confidence.

RL output: size multiplier in [0.5x, 1.5x] applied to a fixed risk budget. **Not before 6+ months of validated outcomes.**

12. Implementation Order

Build in this sequence; each phase is independently shippable and the user can stop at any point with a working subset.

- 1 **Migration** — create the two new tables (positioning_snapshots, positioning_outcomes). New file src-tauri/migrations/0003_positioning.sql. Register in lib.rs.
- 2 **positioning.rs** — new file in src-tauri/src/scoring/ with day-trading positioning logic + pip_size_for() helper.
- 3 **Snapshot writer** — function that calls build_synthesis + derive_positioning and inserts a row. Tauri command force_snapshot(pair).
- 4 **Scheduler** — tokio task in run() that wakes at 18:00 daily, calls snapshot writer for each active pair. Add catch-up logic for app-start-after-18:00.
- 5 **Outcome resolver** — tokio task running hourly. Walks open snapshots, fetches H1 candles, records outcomes. Tauri command force_resolve(id).
- 6 **Reporting commands** — list_resolved_snapshots, get_accuracy_stats. Pure SQL aggregations.

- 7 **UI panel** — new 'AI Positioning' page (or add to Analytics V3 as a new category panel). Render today's snapshot + history table + aggregate tiles. Use the design language from existing panels.
- 8 **Active-pairs config** — small UI in Settings to enable/disable pairs. Default ['EUR/USD'].
- 9 **Wait 60–90 days** — pure observation. Resist the urge to tune anything.
- 10 **Phase A weight tuning** — only once expectancy is positive and out-of-sample holdout confirms.
- 11 **Phase B ML** — only after Phase A is stable for 30+ days.
- 12 **Phase C RL** — research only, much later.

Appendix A — File Paths Reference

Files a future Claude session will need to read or modify.

- `src-tauri/src/lib.rs` — Tauri command registration, app entry point, scheduler spawn.
- `src-tauri/src/scoring/mod.rs` — Synthesis types, `build_synthesis` orchestrator.
- `src-tauri/src/scoring/trend.rs` · `momentum.rs` · `structure.rs` · `volatility.rs` — sub-scorers (read-only for this feature).
- `src-tauri/src/scoring/positioning.rs` — **new file** for day-trading positioning derivation.
- `src-tauri/src/candle_store.rs` — existing candle persistence; `positioning_snapshots/outcomes` follow the same pattern.
- `src-tauri/src/oanda_client.rs` — `fetch_raw_candles_tf(api_key, instrument, gran, count)` for both daily snapshots and H1 outcome resolution.
- `src-tauri/migrations/` — add `0003_positioning.sql` here.
- `src/lib/brain/synthesis.ts` — TS Synthesis types; extend with Positioning/Outcome types matching the Rust additions.
- `src/pages/AnalyticsV3.tsx` — current AI Synthesis panel; the new positioning UI either lives here or in a new page `src/pages/AIPositioning.tsx` wired via the sidebar (`src/components/layout/Sidebar.tsx`).
- `docs/Analytics_PDF.pdf` — indicator reference (for context on what features are available).
- `CLAUDE.md` — architecture rules. Re-read before every session.

Appendix B — Build & Verify Commands

Rust compile check (fast. no link)

```
cargo check --manifest-path src-tauri/Cargo.toml
```

Full dev run (Tauri + Vite)

```
npm run tauri dev
```

TypeScript strict check

```
npx tsc --noEmit
```

Geoff has confirmed: the Tauri app crashes under browser preview — do not use `preview_start` or any in-browser verification. `npm run tauri dev` is the only supported run path. Vite hot-reloads TS changes within ~1s; Rust changes trigger an automatic recompile + relaunch on file save.

Appendix C — Open Decisions for Future Sessions

Items deliberately left open at document creation time. Pick these up with Geoff before committing.

- **Local timezone** for the 18:00 schedule — assume America/Toronto unless Geoff specifies. Use `chrono-tz` if needed.
- **Position sizing** — this plan covers entry/SL/TP only. Risk size in dollars/percent per trade is a separate concern; integrate with the broker account info in Settings when ready.
- **Snapshot on weekends** — FX markets close Friday 5pm NY to Sunday 5pm NY. Should Friday's snapshot resolve over the weekend or skip the weekend? Probable answer: skip (no new bars to walk), and Monday's snapshot starts fresh.
- **Holiday handling** — same as weekends; OANDA returns no bars on closed days.

- **Manual override** — should Geoff be able to discard a snapshot (e.g. major news event makes the bias invalid)?
Add a discarded flag to positioning_snapshots if yes.